Contexts and verdicts

What “the price of randomization” means, where Randomized Online-DRL **loses** against the other methods, and what the two verdicts “**surrogate only**” and “**unsafe reduction**” mean in the context in which they are used. Companion to *Randomized_vs_Online_DRL_comparison.pdf*.

1. What section 7 of the comparison PDF explains

Section 7 is titled **“The price of randomization”**. It does not compare the *quality* of the two algorithms; it quantifies the **computing time** that the randomized policy needs, because the randomized policy does far more work per request than the deterministic one.

Why the randomized policy is more expensive

step	Online-DRL	Randomized Online-DRL	cost per request
decide to download	two integer comparisons (T_i, L_i against θM_i)	same counters, but one random draw from $f(\theta) = e^{\theta/M}/(M(e-1))$	O(1)
update the cache state	credit arithmetic on the cached services only (O(|cache|))	one gradient step on the fractional vector p, then the Euclidean projection onto D_{vec} (dual ascent on 4 multipliers)	**O(N)**
choose the cache content	deterministic eviction order	rounding of K paths ($s_k = 1[U_k < p]$) plus a feasibility repair of every violating path	**O(K·N)**
cost per request	13–26 µs	440–1 665 µs	**25–70×**

Measured (exp14, microseconds per request, wall clock of the request loop only):

N (services)	Online-DRL	LL-RC	R3 only	Randomized	factor
20	12.5	14.9	11.8	**483.8**	38.8×
40	19.7	23.3	15.1	**566.5**	28.7×
80	17.6	22.6	22.3	**856.5**	48.8×
160	21.9	29.5	22.1	**1,413.2**	64.6×
320	24.5	33.9	27.1	**1,857.8**	75.8×

The power-law fit gives µs/req ~ $N^{0.52}$ (R^2 = 0.97) for the randomized policy against $N^{0.17–0.32}$ for the deterministic ones, and the cost is essentially **independent of the cache size k** (fit $C^{-0.07}$). In practice: an edge node serving 1 000 requests/s spends ≈ 35 ms of CPU per second with the randomized policy against ≈ 1 ms with the deterministic one. This is a real but acceptable cost – and it is the price paid for the competitive-ratio improvement of section 5 of the comparison document.

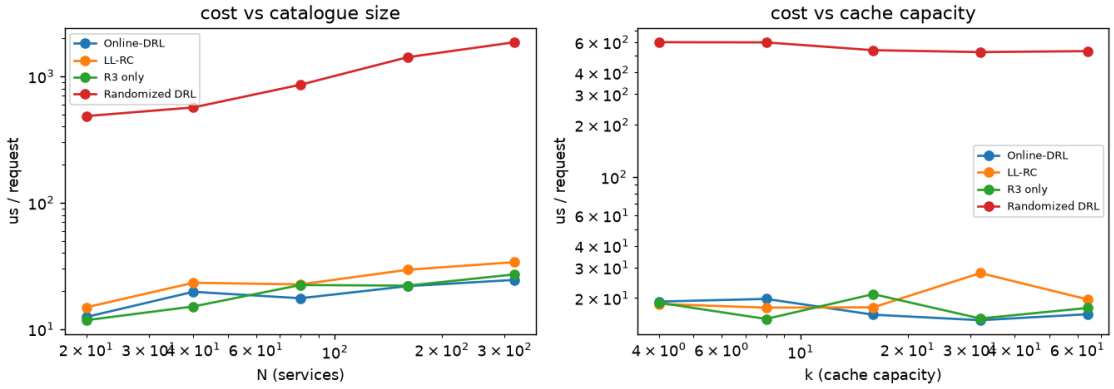


Figure 1 - running time per request against the catalogue size N and the cache size k (exp14). The red curve is the randomized policy; it is ~35x above the deterministic ones and does not depend on k.

2. Does Randomized Online-DRL lose against the other methods?

Yes, in three clearly identifiable situations, and in none of the others. The table below is the complete head-to-head of the four algorithms (plus the exact optimum) over every metric that was measured.

metric	Online-DRL	LL-RC	Randomized DRL	offline OPT	verdict for the randomized DRL
competitive ratio, adversarial family (exp13)	$\gamma = 2.00k$	$\gamma = 1.88k$	$\gamma \approx$ **3 (best)**	$\gamma = 0.40k$	**win**
competitive ratio, Zipf traffic vs MILP (exp13)	1.443	1.457	**1.430 (best)**	-	**win**
competitive ratio, short traces T = 120 (exp10)	1.145 ± 0.072	**1.102 ± 0.058 (best)**	**1.279 ± 0.138 (worst)**	-	**loss**
download cost, same trace (exp13/exp12)	reference	2x det.	**3–13× lower**	2x det.	**win**
latency, multi-resource workload (exp12)	136 226	139 029	**87 333 (-36 %)**	-	**win**
resource violations of the true constraints	0 %	0 %	0 % (after the repair)	-	**tie**
download budget G2	clipped (loses 11.6 % latency)	not enforced	**respected in expectation and per seed**	-	**win**
regret G3, surrogate form (exp09)	-	-	**always below $\sqrt{N}\, l_{max}\, \sqrt{T}$**	-	**win**
regret G3, literal form (exp09)	-	-	**3.7–9.6× the bound, grows with T**	-	**loss**
computing time per request (exp14)	13–26 µs	12–29 µs	**440–1 665 µs (25–70×)**	-	**loss**
warm-up needed before the state is useful	none	none	**yes (visible at T = 120)**	-	**loss**

Summary of the head-to-head. The randomized algorithm wins on the competitive ratio in the regime that matters (adversarial patterns, large caches), wins by a factor 3–13 on download cost, wins on latency on the multi-resource workload, and is the only one of the three that respects a download budget. It loses in exactly three places: (i) on *short* traces it is worse than both deterministic variants (the fractional state is not learned yet), (ii) it needs 25–70× more CPU per request, and (iii) the *literal* form of the proposal's goal G3 is not satisfied. Note also that it never loses on resource feasibility: the repair step makes the constraints exact.

3. What “surrogate only” means (goal G3)

The proposal states a regret goal G3: the algorithm should be close to the best *fixed* fractional cache, with a bound of $\sqrt{N} \cdot l_{max} \cdot \sqrt{T}$. The measurement in exp09 has to decide what is compared with what, and this is where the word **“surrogate”** comes in.

The real quantity we would like to bound

The natural regret is **realized latency**:

$$\Sigma_t\, l_{n_t} - \min_{P \in D_{vec}} \Sigma_t\, l_{n_t}\, 1[n_t \in P] \le \mathrm{Reg}_T$$

i.e. the latency the system really paid minus the latency the best possible *fixed* cache would have paid. The problem is that this expression is **not a function of p alone**: whether a request is a hit, a delayed hit or a forward also depends on the download that may be in flight (the indicator q_t) and on the remaining download time dm_t, both of which are decided by the discrete download dynamics and not by the fractional vector. The bound of Proposition 1 cannot be derived for that expression.

The surrogate that the bound applies to

The proposal replaces the latency by a **surrogate loss** that *is* a function of p:

$$F_t(P) = (1 - p_{n_t}) \cdot g_t, \text{ with } g_t = (1 - q_t) \cdot l_{n_t} + q_t \cdot \min(l_{n_t}, dm_t)$$

It says: *a request that is not covered by the fractional cache costs g_t*. The regret that is actually bounded is therefore

$$\Sigma_t\, F_t(P_t) - \min_{P \in D_{vec}} \Sigma_t\, F_t(P) \le \sqrt{N} \cdot l_{max} \cdot \sqrt{T}$$

because D_{vec} has diameter $\le \sqrt{N}$ and the step size is $\eta = \sqrt{N}/(l_{max}\sqrt{T})$. The comparator $\min_P \Sigma F_t(P)$ is the best **fixed** fractional cache, computed exactly by a linear program (not an approximation).

Measured (exp09):

T	bound $\sqrt{N}\, I_{max}\, \sqrt{T}$	surrogate, stationary	surrogate, peek	surrogate, endogenous	**literal G3**
500	2,970	837	-1,554	-658	**3,912**
1000	4,200	1,199	-2,587	-834	**5,119**
2000	5,940	1,666	-4,480	-898	**6,592**
4000	8,400	2,374	-9,002	-1,172	**9,595**
8000	11,879	3,341	-17,560	-1,496	**12,917**

Reading the table. Every surrogate regret is below the bound (the column "surrogate"), for exogenous and for endogenous gradients, with and without the peek update; the negative values simply mean that the OGD state beats every fixed fractional cache. The last column is the regret computed with the *realized latency* instead of the surrogate: it is 3.7–9.6× the bound and it grows with T. **"Surrogate only"** is the verdict that says: *the statement of G3 holds for the surrogate loss and only for the surrogate loss* - the literal version of the goal is not supported by this implementation.

Why the literal version fails, and what would fix it

The fractional comparator is allowed to **cache a service instantly**, while the real system must first **download** it (paying M_n of cost and of latency) and only then serve it. The comparator is not charged for those downloads, so the gap between the two is at least the cost of the downloads the real algorithm was forced to make. Three possible corrections of the goal: (i) define G3 on the surrogate loss (what the bound actually proves); (ii) charge the comparator for the downloads its fixed cache implies; (iii) compare against a "download-aware" comparator that also has to wait for the service.

4. What "unsafe reduction" means (goal G5)

The proposal replaces the three resource constraints of D_{vec} by a **single** scalar constraint, the so-called **slot reduction**:

$\Sigma_n\, \phi(a_n) \cdot p_n \le 1$ instead of $\Sigma_n\, a_{n,r} \cdot p_n \le 1$ for every r

with a weight function ϕ (dominant resource, a linear or l_p combination of the resources, shadow prices, AHP or entropy weights). The weight function is supposed to be a **safe** surrogate of the true constraints, and the proposal defines the capacity loss it causes as

$\Lambda_\phi = \max_n \phi(a_n) \,/\, \max_r a_{n,r}$ (eq. 18)

$\Lambda_\phi < 1$ means "the reduction charges a service *less* than its true bottleneck demand", i.e. the reduced constraint is **weaker** than the true one, and the cache it produces can exceed the real CPU/RAM/disk limits. Such a reduction is called **unsafe**. The proposal itself warns about it: with weights on the simplex ($\sum_r \alpha_r = 1$) a service that uses only disk looks cheap, and the guarantee is safe only if $\alpha_r \ge 1$ for every r (at the price of wasting up to a factor R of the capacity).

Measured on the same randomized algorithm, same trace, only the reduction changes (exp11, imbalanced workload):

reduction ϕ	Λ_ϕ	latency	U^max	stranded W	max utilisation	**violations of the true resources**
vector	-	91,538	0.723	0.344	0.99	0.000
dominant (max-norm)	1.00	121,403	0.437	0.584	0.73	0.000
linear alpha=1 (safe)	2.52	156,703	0.111	0.190	0.57	0.000
l2-norm alpha=1	1.50	109,668	0.528	0.697	0.65	0.000
linear entropy (simplex)	0.86	82,940	0.963	0.895	1.58	**0.548**
linear AHP (simplex)	0.87	72,308	1.252	1.378	1.70	**0.846**
shadow price (mean rho)	1.00	89,142	1.651	1.854	2.37	**0.842**

Reading the table. The three reductions that *look* attractive - entropy weights, AHP weights and uniform shadow prices - all have $\Lambda_\phi < 1$ and give a *lower* latency than the exact vector formulation, and they are exactly the three that break the real resource limits in 55–85 % of the steps (up to 2.37× the capacity). The safe reductions (dominant resource, l_2-norm, linear with $\alpha = 1$) never violate anything, but they waste capacity: the linear $\alpha = 1$ reduction has $\Lambda_\phi = 2.52$, leaves the bottleneck resource at 11 % utilization and costs 71 % more latency. **"Unsafe reduction"** is therefore the warning that a slot reduction is not a free simplification: unless the weights dominate the true demands, it produces a cache that the edge node physically cannot hold - and the simulation would show the violation as a utilization above 1 on the true resources, not as a slower run.

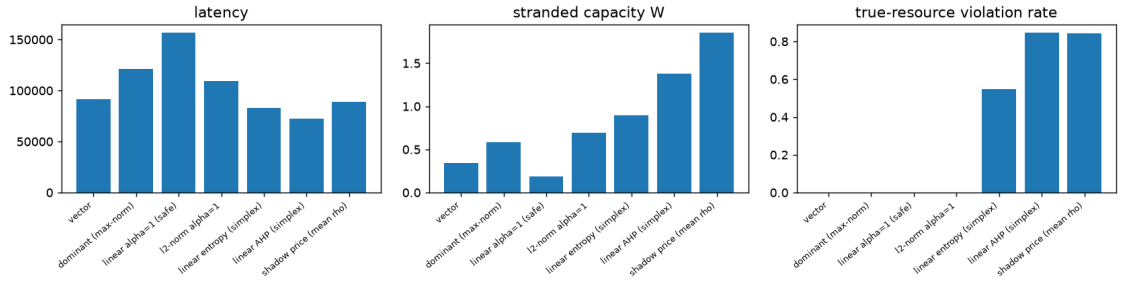


Figure 2 - the same randomized algorithm under different slot reductions (exp11). The unsafe ones (entropy, AHP, shadow price) have a lower latency and violate the true resource limits.

5. Glossary of the terms used in both documents

term	meaning in this context
hit / delayed hit / forwarding	the three ways a request is served: from the cache (latency 0), buffered while a download finishes (latency min(l, remaining)), or forwarded to the cloud (latency l)
latency vs. cost	latency is what the user waits; cost is the download time the operator pays. They are optimized separately and never mixed in a metric
miss epoch, T_i, L_i	the period since the service was last cached; T_i = elapsed time, $L_i = l_n \times$ number of requests of the service in the epoch. The download trigger compares them with θM_i
credit (Landlord)	a real-valued budget attached to every cached service, initialised to M_f and reset on every hit; it reaches 0 and the service becomes evictable
D_{vec}	the polytope of feasible fractional caches: $p \in [0,1]^N$ with $\Sigma p_n \le C$ and $\Sigma a_{n,r} p_n \le 1$ for every resource r
projected OGD	one gradient step on the requested service followed by a projection back onto D_{vec}; the gradient is the surrogate gradient g_t
rounding / repair	the fractional p is turned into a boolean cache by thresholding K persistent uniform vectors; a path that violates the constraints is repaired by evicting services until it fits
q_t, dm_t	$q_t = 1$ if a download of the requested service is in flight; dm_t = its remaining time. Together they decide hit vs. delayed hit vs. forwarding and they enter the surrogate gradient
peek	update the state with the gradient of request t *before* serving it (the proposal's variant) as opposed to after (textbook OGD)
γ (competitive ratio)	ALG(I)/OPT(I): the latency of the algorithm divided by the latency of the offline optimum (the MILP, or the paper's (k+2n)l on the Large Sequence)
β_{up}, β_{down}	the fraction of the download slots during which a request is forwarded instead of buffered; β_{up} = max over services (used with M_x), β_{down} = mean (used with M_{min})
$U_{r,t}$, U^max, imbalance, stranded W	per-resource occupancy, its maximum over the resources, the spread between the resources, and the capacity of the bottleneck resource that the other resources cannot use
Λ_ϕ, Λ■	the theoretical capacity loss of a reduction (eq. 18) and its empirical estimate E[U^max]/E[mean$_r$U$_r$]
surrogate	a differentiable, p-dependent stand-in for the discrete latency; the quantity the regret bound applies to
unsafe reduction	a slot reduction whose weights under-charge the true demands ($\Lambda_\phi < 1$), so the resulting cache can exceed the real resource limits

6. How each metric in the comparison is calculated

metric	formula / procedure	why it is the right one
latency	Σ_t lat$_t$ with lat$_t$ = 0 (hit), min(l_n, remaining) (delayed hit) or l_n (forward)	exactly the objective of the paper
cost	ΣM_n over the *started* downloads	the operator's cost; forwarding is charged through the latency, not the cost
downloads	number of started downloads	the budget is spent per download
competitive ratio γ	ALG(I)/OPT(I), OPT from the MILP (HiGHS) or (k+2n)l on the Large Sequence	the paper's definition; status = 1 would mean the MILP hit the time limit, then the ratio is only a lower bound
fit of γ(k)	least squares: linear, logarithmic, power law (log-log), with R^2	decides empirically whether the linear behaviour of Theorem 1 disappears; it is not a proof
$U_{r,t}$	$\Sigma_{n \text{ cached at } t} a_{n,r}$	the proposal's per-resource occupancy
imbalance, stranded W	U^max – min$_r$U$_r$ and Σ_r(U^max – U$_r$)	proposal eqs. (30)-(33)

metric	formula / procedure	why it is the right one
violation rate	fraction of steps with $\max_r U_{r,t} > 1 + 1e\text{-}9$	goal G1 is about the *true* constraints, not the reduced ones
Λ_ϕ	$\max_n \phi(a_n)/\max_r a_{n,r}$	proposal eq. (18)
surrogate regret	$\Sigma_t F_t(P_t) - \min_P \Sigma_t F_t(P)$, the minimum computed exactly by an LP	the comparator of Proposition 1, solved to optimality
β_{up}, β_{down}	per service fwd_i/M_i, where fwd_i counts the requests forwarded while a download of i was in flight	the paper's beta is exactly that fraction
running time	$10^6 \cdot$wall-clock time of the request loop / T	measures the algorithm only, not the catalogue or the trace
latency per unit resource	L / $\Sigma_t U\text{^max}_t$	proposal eq. (34): latency normalized by the bottleneck resource actually used

All numbers are reproducible with **python experiments/run_all.py**; the raw values are in **results/*.json** and the figures in **results/*.png**. The companion document *Randomized_vs_Online_DRL_comparison.pdf* contains the same data in a shorter, more graphical form, and *docs/simulation_methodology.md* the full description of the model.